

Submission Worksheet

Submission Data

Course: IT114-450-M2025

Assignment: IT114 Milestone 2 - RPS

Student: Mariano R. (mr822)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 7/21/2025 3:58:11 PM

Updated: 7/21/2025 11:55:49 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-2-rps/grading/mr822>

View Link: <https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-2-rps/view/mr822>

Instructions

1. Refer to Milestone2 of [Rock Paper Scissors](#)
 1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the Milestone2 branch
 1. `git checkout Milestone2`
 2. `git pull origin Milestone2`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 1. `git add .`
 2. ``git commit -m "adding PDF"`
 3. `git push origin Milestone2`
 4. On Github merge the pull request from Milestone2 to main
7. Upload the same PDF to Canvas
8. Sync Local
 1. `git checkout main`
 2. `git pull origin main`

Section #1: (1 pt.) Payloads

Progress: 100%

≡ Task #1 (1 pt.) - Show Payload classes and subclasses

Progress: 100%

Details:

- Reqs from the document
 - Provided Payload for applicable items that only need client id, message, and type

- PointsPayload for syncing points of players
- Each payload will be presented by debug output (i.e, properly override the toString() method like the lesson examples)

Part 1:

Progress: 100%

Details:

- Show the code related to your payloads (Payload, PointsPayload, and any new ones added)
- Each payload should have an overridden toString() method showing its internal data

SS Code 1

```
Common > J PointsPayload.java > ...
1 // UCID: mrmoms2004
2 // Date: 07/21/2022
3 // Description: Extends Payload to include points, from system (pgm) session.
4
5 package Project.Common;
6
7 public class PointsPayload extends Payload {
8     private int points;
9
10    public int getPoints() {
11        return points;
12    }
13
14    public void setPoints(int points) {
15        this.points = points;
16    }
17
18    @Override
19    public String toString() {
20        return String.format("PointsPayload[%s] Client Id [%s] Points: [%d]",
21            getPayloadType(), getClientId(), points);
22    }
23
24 }
```

SS Code 2



Saved: 7/21/2025 4:01:29 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the purpose of each payload shown in the screenshots and their properties

Your Response:

Payload.java is the base class for messages, containing type, client ID, and message text. **PointsPayload.java** extends it to include player points for syncing scores during the game.

Section #2: (4 pts.) Lifecycle Events

Progress: 100%

Task #1 (0.80 pts.) - GameRoom Client Add/Remove

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the `onClientAdded()` code
- Show the `onClientRemoved()` code

```
@Override
protected synchronized void addClient(ServerThread client) {
    super.addClient(client); // Call Room logic
    if (!activePlayers.contains(client)) {
        activePlayers.add(client);
    }
    broadcastToActivePlayers(String.format("Player %s joined the game.", client.getClientName()));
}

@Override
protected synchronized void removeClient(ServerThread client) {
    super.removeClient(client); // Call Room logic
    activePlayers.remove(client);
    broadcastToActivePlayers(String.format("Player %s left the game.", client.getClientName()));
}

private void broadcastToActivePlayers(String message) {
    for (ServerThread client : activePlayers) {
        client.sendMessage(client.getClientId(), message);
    }
}
```

SS Code

Part 2:

Progress: 100%

Details:

- Briefly note the actions that happen in `onClientAdded()` (app data should at least be synchronized to the joining user)
- Briefly note the actions that happen in `onClientRemoved()` (at least should handle logic for an empty session)

Your Response:

`addClient()` adds the player to the room and the active players list, syncs existing players, and broadcasts that they joined. While `removeClient()` removes the player, updates the active list, and broadcasts that they left.

☰ Task #2 (0.80 pts.) - GameRoom Session Start

Progress: 100%

Details:

- Reqs from document
 - First round is triggered
- Reset/set initial state

📎 Part 1:

Progress: 100%

Details:

- Show the snippet of `onSessionStart()`

```
protected synchronized void onSessionStart() {  
    roundNumber = 1;  
    eliminatedPlayers.clear();  
    broadcastToActivePlayers("Game started! Round 1 begins now.");  
    onRoundStart();  
}
```

SS Code



Saved: 7/21/2025 11:45:11 PM

☰ Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up initial session state for your project) and next lifecycle trigger

Your Response:

This method sets up the initial game session state by resetting the round number to 1 and clearing any previously eliminated players. It also notifies all active players that the game has started and that Round 1 is beginning. The next lifecycle trigger would typically involve prompting each player to make their move for the round (e.g., Rock, Paper, or Scissors).



Saved: 7/21/2025 11:45:11 PM

☰ Task #3 (0.80 pts.) - GameRoom Round Start

Progress: 100%

Details:

- Reqs from Document
 - Initialize remaining Players' choices to null (not set)
 - Set Phase to "choosing"
 - GameRoom round timer begins

🖼 Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundStart()`

```
protected synchronized void onRoundStart() {  
    currentPhase = "choosing";  
    playerChoices.clear();  
  
    for (ServerThread player : activePlayers) {  
        playerChoices.put(player, null);  
    }  
  
    broadcastToActivePlayers(String.format("Round %d started. Please submit your move!", roundNumber));  
}
```

SS Code

📁 Saved: 7/21/2025 8:17:44 PM

☰ Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., setting up the round for your project)

Your Response:

This method sets up a new round by clearing previous choices, setting all active players' choices to null, changing the phase to "choosing", and prompting all players to submit their moves for the round.

📁 Saved: 7/21/2025 8:17:44 PM

☰ Task #4 (0.80 pts.) - GameRoom Round End

Details:

- Reqs from Document
 - **Condition 1:** Round ends when round timer expires
 - **Condition 2:** Round ends when all active Players have made a choice
 - All Players who are not eliminated and haven't made a choice will be marked as eliminated
 - Process Battles:
 - Round-robin battles of eligible Players (i.e., Player 1 vs Player 2 vs Player 3 vs Player 1)
 - Determine if a Player loses if they lose the "attack" or if they lose the "defend" (since each Player has two battles each round)
 - Give a point to the winning Player
 - Points will be stored on the Player/User object
 - Sync the points value of the Player to all Clients
 - Relay a message stating the Players that competed, their choices, and the result of the battle
 - Losers get marked as eliminated (Eliminated Players stay as spectators but are skipped for choices and for win checks)
 - Count the number of non-eliminated Players
 - If one, this is your winner (onSessionEnd())
 - If zero, it was a tie (onSessionEnd())
 - If more than one, do another round (onRoundStart())

Part 1:

Progress: 100%

Details:

- Show the snippet of `onRoundEnd()`

```

1 // Round ends when round timer expires
2 // Round ends when all active Players have made a choice
3 // All Players who are not eliminated and haven't made a choice will be marked as eliminated
4 // Process Battles:
5 // Round-robin battles of eligible Players (i.e., Player 1 vs Player 2 vs Player 3 vs Player 1)
6 // Determine if a Player loses if they lose the "attack" or if they lose the "defend" (since each Player has two battles each round)
7 // Give a point to the winning Player
8 // Points will be stored on the Player/User object
9 // Sync the points value of the Player to all Clients
10 // Relay a message stating the Players that competed, their choices, and the result of the battle
11 // Losers get marked as eliminated (Eliminated Players stay as spectators but are skipped for choices and for win checks)
12 // Count the number of non-eliminated Players
13 // If one, this is your winner (onSessionEnd())
14 // If zero, it was a tie (onSessionEnd())
15 // If more than one, do another round (onRoundStart())

```

SS Code 1

```

1 // Round ends when round timer expires
2 // Round ends when all active Players have made a choice
3 // All Players who are not eliminated and haven't made a choice will be marked as eliminated
4 // Process Battles:
5 // Round-robin battles of eligible Players (i.e., Player 1 vs Player 2 vs Player 3 vs Player 1)
6 // Determine if a Player loses if they lose the "attack" or if they lose the "defend" (since each Player has two battles each round)
7 // Give a point to the winning Player
8 // Points will be stored on the Player/User object
9 // Sync the points value of the Player to all Clients
10 // Relay a message stating the Players that competed, their choices, and the result of the battle
11 // Losers get marked as eliminated (Eliminated Players stay as spectators but are skipped for choices and for win checks)
12 // Count the number of non-eliminated Players
13 // If one, this is your winner (onSessionEnd())
14 // If zero, it was a tie (onSessionEnd())
15 // If more than one, do another round (onRoundStart())

```




SS Code

 Saved: 7/21/2025 9:09:06 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the logic that occurs here (i.e., cleanup/reset, next lifecycle events)

- Details:**

 - Briefly explain the logic that occurs here (i.e., cleanup/reset, next lifecycle events)

Your Response:

At the end of the session, the server broadcasts a "Game over!" message and sends all players a final scoreboard sorted by points. It then resets each player's server-side data (like `clientId` and `name`) and triggers a client-side user list reset, preparing them for a new game. The round-related state variables are also cleared, and the game is left in a waiting state until a new ready check starts the next session.

 Saved: 7/21/2025 9:09:06 PM

Section #3: (4 pts.) Gameroom User Action And State

Progress: 100%

≡ Task #1 (2 pts.) - Choice Logic

Progress: 100%

- Reqs from document
 - Command: `/pick <[r,p,s]>` (user picks one)
 - GameRoom will check if it's a valid option
 - GameRoom will record the choice for the respective Player
 - A message will be relayed saying that "X picked their choice"
 - If all Players have a choice the round ends

- Reqs from document
 - Command: `/pick <[r,p,s]>` (user picks one)
 - GameRoom will check if it's a valid option
 - GameRoom will record the choice for the respective Player
 - A message will be relayed saying that "X picked their choice"
 - If all Players have a choice the round ends

Details:

- Show the code snippets of the following, and clearly caption each screenshot
- Show the Client processing of this command (process client command)
- Show the ServerThread processing of this command (process method)
- Show the GameRoom handling of this command (handle method)
- Show the sending/syncing of the results of this command to users (send/sync method)
- Show the ServerThread receiving this data (send method)
- Show the Client receiving this data (process method)

```
// UCID: mramos2001
// Date: 07/21/2025
// Description: Send RPS round results to all players
private void sendRoundResults(Map<ServerThread, Integer> roundPoints) {
    for (Map.Entry<ServerThread, Integer> entry : roundPoints.entrySet()) {
        ServerThread player = entry.getKey();
        int pointsEarned = entry.getValue();
        String summary = "You earned " + pointsEarned + " point" + (pointsEarned != 1 ? "s." : ".");
        player.sendMessage(Constants.DEFAULT_CLIENT_ID, summary); // Message to player
    }
}
```

SS Code 4

```
// UCID: mramos2001
// Date: 07/21/2025
/**
 * Sends a message to the client
 *
 * @param clientId who it's from
 * @param message
 * @return true for successful send
 */
protected boolean sendMessage(long clientId, String message) {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.MESSAGE);
    payload.setMessage(message);
    payload.setClientId(clientId);
    return sendToClient(payload);
}
```

SS Code 5

```
break;
case MESSAGE:
    processMessage(payload);
    break;
case REVERSE:
```

SS Code 6

```
@Override
public synchronized void handleChoice(ServerThread player, String choice) {
    if (choice.equalsIgnoreCase("quit")) {
        player.sendMessage(Constants.DEFAULT_CLIENT_ID, "You can't pick right now.");
        return;
    }
    if (!isValidChoice(choice)) {
        player.sendMessage(Constants.DEFAULT_CLIENT_ID, "Invalid choice. Use rock, paper, or scissors.");
        return;
    }
    if (!player.isInGame()) {
        player.sendMessage(Constants.DEFAULT_CLIENT_ID, "You're not part of this round.");
    }
}
```

```

    if (playerChoices.get(player) != null) {
        player.sendMessage(Constants.DEFAULT_CLIENT_ID, "You already picked.");
        return;
    }

    playerChoices.put(player, choice.toLowerCase());
    broadcastToActivePlayers(player.getClientName() + " has picked.");

    if (allChoicesSubmitted()) {
        endRound();
    }
}

```

SS Code 3

```

// UCID: mramos2001
// Date: 07/21/2025
// Description: Add support for /pick command and CHOICE payload to Client.java
else if (text.startsWith(Command.PICK.command)) {
    text = text.replace(Command.PICK.command, "").trim();

    if (!text.equalsIgnoreCase("rock") && !text.equalsIgnoreCase("paper") && !text.equalsIgnoreCase("scissors")) {
        LoggerUtil.INSTANCE.warning(LoggerUtil.format("Invalid choice: the /pick only accepts rock, paper, or scissors.", text, RH));
        return true;
    }

    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.CHOICE);
    payload.setChoice(text.toLowerCase());
    sendToServer(payload);

    wasCommand = true;
}

else if (text.startsWith(Command.START.command)) {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.START);
    sendToServer(payload);
    wasCommand = true;
}

```

SS Code 1

```

        break;
// UCID: mramos2001
// Date: 07/21/2025
// Description: Handle CHOICE payload from client
case CHOICE:
    if (currentRoom != null) {
        currentRoom.handleChoice(this, incoming.getChoice());
    }
    break;
case START:
    if (currentRoom instanceof GameRoom gameRoom) {
        gameRoom.onSessionStart();
    }
    break;

```

SS Code 2

 Saved: 7/21/2025 11:42:16 PM

Part 2:

Progress: 100%

Details:

- Briefly explain/list in order the whole flow of this command being handled from the client-side to the server-side and back

Your Response:

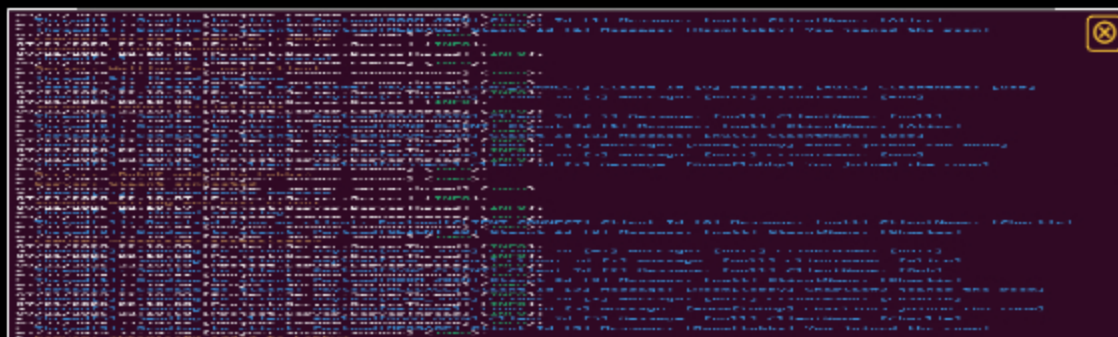
The /pick command is typed by the client, which creates and sends a CHOICE payload to the server. The server's ServerThread routes it to the GameRoom, which saves the player's choice, broadcasts a confirmation, and if all players have picked, it ends the round. The round logic determines outcomes, assigns points, sends result messages back to clients, and clients display those using processChoice and processMessage.

 Saved: 7/21/2025 11:42:16 PM

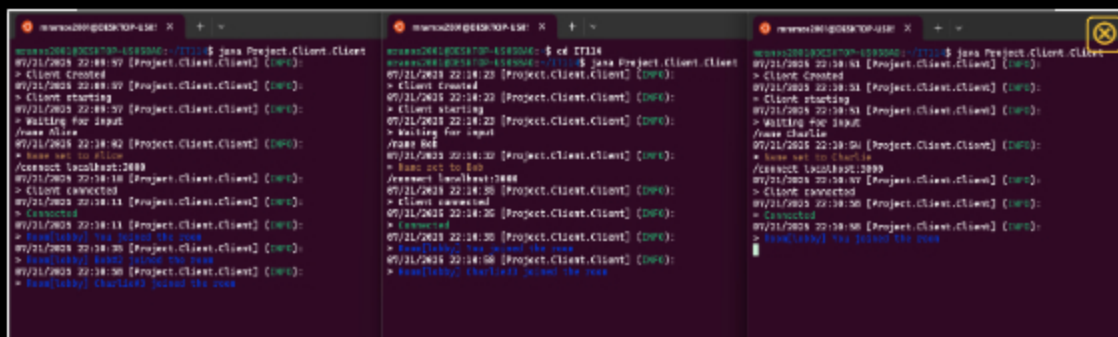
Progress: 100%

Details:

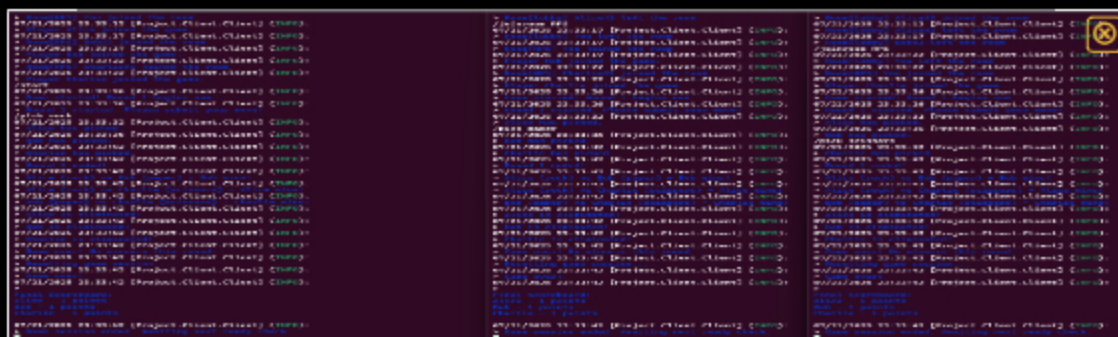
- Show examples from the terminal of a full session demonstrating each command and progress output
- This includes battle outcomes, scores and scoreboards, etc
- Ensure at least 3 Clients and the Server are shown
- Clearly caption screenshots



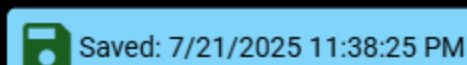
SS Server



SS Connections



SS Game



Section #4: (1 pt.) Misc

☰ Task #1 (0.33 pts.) - Github Details

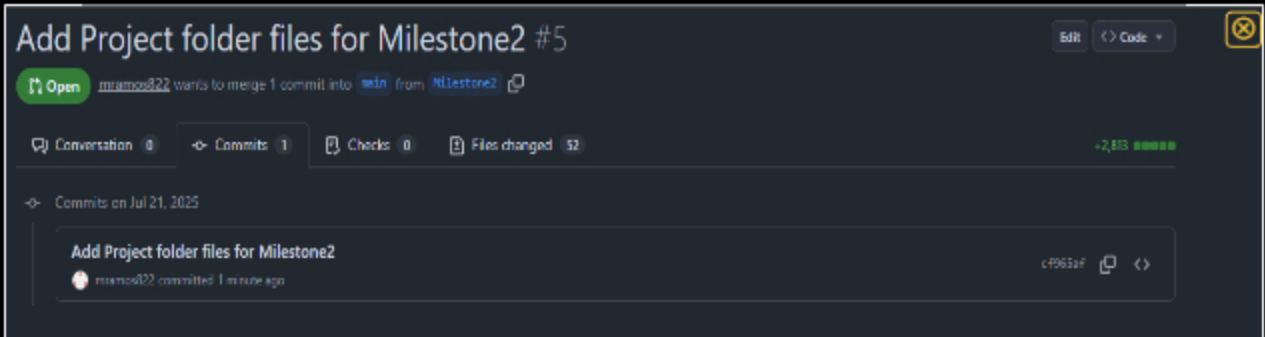
Progress: 100%

🖼️ Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history



Screenshot



Saved: 7/21/2025 11:55:49 PM

🔗 Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`)

URL #1

<https://github.com/mramos822/mr822-IT1144156/>



URL

<https://github.com/mramos822/n>



Saved: 7/21/2025 11:55:49 PM

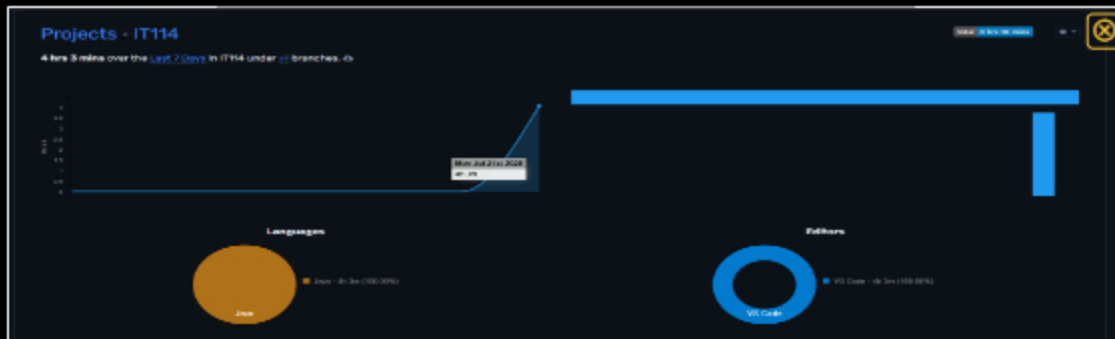
🖼️ Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

necessary



SS 1

Files		Branches	
1 hr 38 mins	Server/RoomRoom.java	4 hrs 3 mins	Milestone2
1 hr 15 mins	Client/Client.java	1 sec	main
40 mins	Server/ServerThread.java		
9 mins	Server/Server.java		
6 mins	Server/Room.java		
6 mins	Common/PayloadType.java		
5 mins	Common/Payload.java		
2 mins	Common/Command.java		
45 secs	Common/User.java		
11 secs	Server/RoomNotFoundException.java		
11 secs	Common/ConnectionPayload.java		
9 secs	Common/RoomAction.java		
1 sec	Common/Constants.java		
0 secs	Client/ClientLClass		
0 secs	Common/PointPayload.java		

SS 2



Saved: 7/21/2025 11:54:38 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned how to build a multiplayer networked game using Java sockets and multi-threading. The project helped me understand synchronization, client-server communication, and managing game state across multiple players. I also improved my skills in debugging and using Git for version control.



Saved: 7/21/2025 11:55:11 PM

Task #2 (0.33 pts.) - What was the easiest part of the

assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was implementing the basic client-server communication and sending simple messages between them. Setting up the connection and exchanging payloads was straightforward thanks to the clear protocols we designed.



Saved: 7/21/2025 11:55:27 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was implementing the game logic for the Rock-Paper-Scissors rounds, especially managing player choices, eliminations, and ensuring all clients stayed synchronized during the game cycle.



Saved: 7/21/2025 11:55:39 PM